The phase of the 3-pin electrical socket is connected to normally open (NO) pin of the relay output pin. The neutral from the mains is connected to common (COM) pin of the relay module. An additional 6A SPST electrical switch (S1), connected across NO and normally closed (NC) terminals, acts as a bypass switch when smart switching action is not needed.

The Table shows pin connections between Arduino Uno board and various modules in the project. Arduino Uno is a microcontroller board based on 8-bit ATmega328P microcontroller, which is very simple to use and interface. Use a 12V, 1A SMPS power supply to power up the Arduino Uno board.

Fig. 7: 5V single-channel relay module

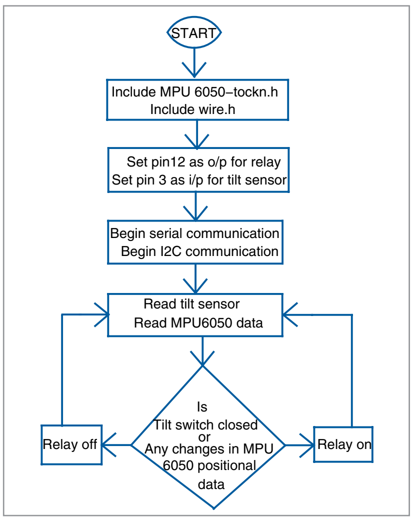

Fig. 8: Flow chart of smart switch box program

ARDUINO UNO PIN CONNECTIONS

Arduino Uno Pins	Components
5V	• Connected to MPU6050 module Vcc pin • Connected to Vcc pin of relay board • Connected to mercury tilt switch pin 2
Pin 2	MPU6050 module INT pin
Pin 3	Mercury tilt switch output pin 3
A4	SDA pin of MPU6050 module
A5	SCL pin of MPU6050 module
Pin 12	Input signal pin of relay board
GND	• Connected to GND pin of relay board • Connected to MPU6050 module GND pin • Connected to mercury tilt switch module GND pin

The software

The circuit operation is performed through the software program loaded into the internal memory of Arduino Uno. The program/sketch (*ironbox.ino*) is written in Arduino programming language. ATmega328P on Arduino Uno comes with a pre-programmed boot loader that allows users to upload a new code to it without using a peripheral hardware programmer.

Arduino IDE is used to compile and upload the program. The sketch is the heart of the system and carries out all major functions. The program is easy and simple to understand. Comments are given at the end of each command line. Fig. 8 shows the flow chart of the program.

When input and output pins of Arduino board are initialised, the following functions and libraries are used.

Serial.begin(). Establishes serial communication between Arduino Uno board and another device via a USB cable. It permits the two devices to communicate using a serial protocol. Here serial communication is used only to read the data from the MPU6050 via Arduino Uno onto the screen. The displayed data allows us to know the changes in the values during movement of the iron box. Sample values are shown in Fig. 5.

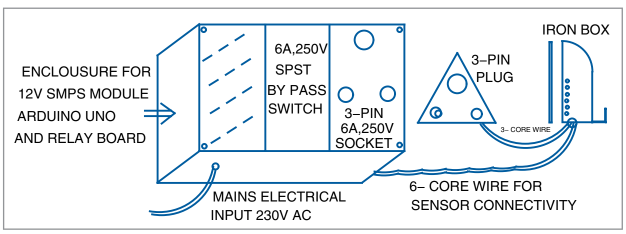

Fig. 9: Proposed construction details

#include <Wire.h>. The Wire library allows communication through I2C devices, also called '2 wire' or 'TWI' (two-wire interface). The SDA (data line) and SCL (clock line) are used for communicating with MPU6050 module.

#include <MPU6050_tockn.h>. It is the Arduino library for easy communication with the MPU6050 module. It can be downloaded from the *link https://github.com/tockn/MPU6050_tockn*

mpu6050.update(). We must execute *mpu6050.update()* to fetch all the data from MPU6050.

There are some important variables in the code. The '*value_1*' variable sums up the angle values in the three directions at a time and passes the absolute value (negative sign is not considered). Similarly, '*value_2*' variable sums up the angle values in the three directions at another point of time and passes the absolute value.

The '*diff*' variable contains the difference of *value_1* and *value_2*. When iron box is stationary, the *diff* value will be zero or close to zero. Based on the value in the *diff* variable, the movement of the iron box is detected. The '*i*' value is used to provide some delay to switch off the relay after no movement of the iron box is detected.

Construction and testing

Fix a 6A, 250V electrical socket on top of a suitable plastic box. Enclose the whole circuit including Arduino